

RAICES DE ECUACIONES

Metodos Numericos IOC 3205 · Capítulos 5 y 6 · Formulario de laboratorio

1 EL PROBLEMA

Buscar el valor de x donde la funcion vale cero. Ese valor es la **raiz**. Toda ecuacion se reescribe igualada a cero:

$$x = \cos(x) \rightarrow f(x) = x - \cos(x) = 0$$

En problemas aplicados: $f = \text{modelo} - \text{valor de diseno}$. Ej: piden $Q_d = 0,05 \rightarrow f(\theta) = Q(\theta) - 0,05$

Se resuelve por **iteracion**: se parte de una estimacion y se mejora hasta que el error baja de una tolerancia.

2 LAS DOS FAMILIAS (pregunta clasica de prueba)

	CERRADOS (bracketing)	ABIERTOS
Necesitan	un intervalo $[x_l, x_u]$ que encierre la raiz	uno o dos puntos, sin encerrar la raiz
Convergen	SIEMPRE (garantia)	PUEDEN DIVERGIR
Velocidad	lentos	mucho mas rapidos
Cuales	biseccion, falsa posicion	punto fijo, Newton, secante

El intercambio es siempre el mismo: seguridad contra velocidad.

3 NOTACION

Simbolo	Del ingles	Que es
x_l	lower	extremo inferior del intervalo (izquierda)
x_u	upper	extremo superior del intervalo (derecha)
x_r	root	estimacion de la raiz en esa iteracion
$f(x_l)$		la funcion evaluada en x_l = altura de la curva ahi

4 BISECCION

Se apoya en: teorema del valor intermedio. Si f es continua y cambia de signo en los extremos, hay raiz adentro.

$$f(x_l) \cdot f(x_u) < 0 \rightarrow \text{hay raiz en el intervalo}$$

Algoritmo:

- $x_r = (x_l + x_u) / 2$ (punto medio)
- evaluar $f(x_l) \cdot f(x_r)$:
 $< 0 \rightarrow$ raiz en la mitad izquierda $\rightarrow x_u = x_r$
 $> 0 \rightarrow$ raiz en la mitad derecha $\rightarrow x_l = x_r$
- repetir hasta que el error baje de la tolerancia

El error se conoce ANTES de empezar (unico metodo que lo permite). Cada paso bota la mitad del intervalo:

$$E_n = \Delta x / 2^n \quad n = \log_2(\Delta x / E_{\text{deseado}})$$

Ej: intervalo $[2,6]$, ancho 4 $\rightarrow$ tras 3 iteraciones mide $4/2^3 = 0,5$. El ancho del intervalo ES tu error maximo.

Convergencia lineal: gana ~0,3 digitos por iteracion. Bajar el error 10 veces cuesta ~3,3 iteraciones.

Trampa: si el intervalo tiene un numero PAR de raices no hay cambio de signo y el test falla. Graficar siempre primero.

5 FALSA POSICION

Critica a biseccion: parte por la mitad aunque un extremo este muchisimo mas cerca de cero. Falsa posicion une los extremos con una **recta** y toma donde esa recta cruza el eje.

$$x_r = x_u - f(x_u) \cdot (x_l - x_u) / (f(x_l) - f(x_u))$$

El resto del algoritmo es **identico** a biseccion (mismo test de signos). Solo cambia como se calcula x_r .

Falla con funciones muy curvadas: un extremo queda pegado y el intervalo no se achica (caso clasico $x^4 - 1$ en $[0, 1,3]$). Por eso el ancho ya no sirve como error: usar ϵ entre x_r sucesivos.

6 NEWTON-RAPHSON

Traza la **recta tangente** en un punto y toma donde esa tangente cruza el eje.

$$x_{i+1} = x_i - f(x_i) / f'(x_i)$$

De donde sale (piden justificarlo): es la serie de Taylor cortada en el termino lineal.

$$f(x_{i+1}) \approx f(x_i) + f'(x_i)(x_{i+1} - x_i) \quad \text{se impone } f(x_{i+1}) = 0 \text{ y se despeja.}$$

Convergencia cuadratica: $E_{i+1} \approx -f''/(2f') \cdot E_i^2$. El numero de cifras correctas **se duplica** cada iteracion: 2, 4, 8, 16. Converge en 3 o 4 pasos.

Los 4 modos de falla: punto de inflexion cerca de la raiz · oscilacion en torno a un maximo o minimo local · arranque cerca de un extremo ($f' \approx 0$, la tangente casi horizontal la dispara) · $f'(x)=0$ exacto, division por cero.

7 SECANTE

Newton para cuando NO tienes la derivada: la aproxima con dos puntos ya calculados.

$$x_{i+1} = x_i - \frac{f(x_i) \cdot (x_{i-1} - x_i)}{(f(x_{i-1}) - f(x_i))}$$

Formula identica a falsa posicion, pero: falsa posicion es CERRADA (elige que punto reemplazar segun el signo) y secante es ABIERTA (usa siempre los dos ultimos, puede diverger). *Esto se pregunta.*

Orden 1,618 (numero aureo). Mas lenta que Newton (2), mas rapida que biseccion (1). Solo 1 evaluacion de f por iteracion contra 2 de Newton.

8 TABLA COMPARATIVA

Metodo	Familia	Necesita	Orden	Garantia
Biseccion	cerrado	intervalo con cambio de signo	1	SI
Falsa posicion	cerrado	intervalo con cambio de signo	1 (mas rapido)	SI
Punto fijo	abierto	un x_0 y un despeje $x = g(x)$	1	solo si $ g' < 1$
Newton	abierto	un x_0 + la derivada f'	2	NO
Secante	abierto	dos puntos (sin derivada)	1,618	NO

9 EN PYTHON: scipy.optimize.root_scalar

Regla de oro: tengo intervalo → brentq | tengo derivada → newton | no tengo nada → secant

```
from scipy.optimize import root_scalar
import numpy as np ; import matplotlib.pyplot as plt
def f(x):
    return modelo(x) - objetivo
xl, xu = a, b
print(f(xl)*f(xu)) # DEBE SER NEGATIVO
R = root_scalar(f, method='brentq', bracket=(xl,xu))
R = root_scalar(f, method='bisect', bracket=(xl,xu), xtol=Es, maxiter=k)
R = root_scalar(f, method='newton', x0=p0, fprime=df)
R = root_scalar(f, method='secant', x0=p0, x1=0.9*p0)
print(R.root, R.iterations, R.converged, R.flag)
```

Salida RootResults: .root · .converged · .flag · .iterations · .function_calls

Lo que pide el codigo es lo que el metodo necesita en la teoria: cerrados piden bracket, Newton pide fprime, secante pide dos puntos.

10 ITERACIONES DE BISECCION EN CODIGO

```
Es = 0.5 * eps_s * abs(wl + wu) # eps_s = 0.5e-5 -> 5 cifras signif.
k = int(log2((wu - wl)/Es)) + 1 # el +1 hace de techo
```

OJO: si $wl = 0$ entonces $Es = 0$ y bisect lanza error. Correr el intervalo ($wl = 0.01$).

11 VERIFICACIONES QUE DAN PUNTOS

• cambio de signo antes de llamar · evaluar $f(\text{raiz}) \approx 0$ · revisar `.converged == True` · sentido fisico del resultado (ej: $h/D < 0,5$ = tuberia bajo la mitad; angulo entre 10° y 60°)

12 ERRORES FRECUENTES

Mensaje / sintoma	Que pasa
$f(a)$ and $f(b)$ must have different signs	el bracket no encierra raiz. Graficar y reelegir.
Failed to converge	subir maxiter o mejorar el punto inicial
converged = False	el valor devuelto NO tiene sentido: es basura convincente
TypeError en args	args va como TUPLA: <code>args=(a,b)</code> , no lista
newton sin fprime	scipy usa diferencias finitas: mas lento y menos preciso
<code>math.exp</code> / <code>math.cos</code> con <code>linspace</code>	no operan sobre arrays: usar <code>[f(t) for t in x]</code> o <code>numpy</code>

Referencia medida (tuberia de Manning, mismo problema): biseccion 41 iteraciones · brentq 10 · toms748 5. Misma raiz con 6 cifras significativas.